МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

Факультет информационных технологий
Кафедра «Инфокогнитивные технологии»

КУРСОВОЙ ПРОЕКТ

на тему: *«Разработка клиент-серверного desktop-приложения для поддержки бизнес-процессов компании»»*
Направление подготовки 09.03.03 «Прикладная информатика»
Профиль «Разработка и интеграция бизнес-приложений»

Выполнила:

Андрианова Мария Ивановна

Москва 2025

Введение

Страховая компания представляет собой крупную сеть филиалов, расположенных по всей территории страны, каждый из которых обладает собственным наименованием, адресом и контактным телефоном. В рамках своей деятельности компания оказывает услуги различным лицам, о которых сохраняется информация о фамилии, имени, отчестве, дате рождения и номере телефона. Заключает страховые договоры, охватывающие самые разные объекты и риски: от страхования автотранспорта от угона и страхования жилой недвижимости до добровольного медицинского страхования. Каждый договор сопровождается фиксацией основных параметров — даты заключения, страховой суммы, вида страхования, применяемой тарифной ставки и филиала, в котором он был оформлен. Оформление и сопровождение договоров возлагается на страховых агентов, для которых ведётся учёт полного комплекса персональных данных, включая фамилию, имя, отчество, адрес и номер телефона, а также информацию о том, в каком филиале они работают. Заработная плата каждого агента рассчитывается в виде процента от страхового платежа, представляющего собой произведение страховой суммы на тарифную ставку, при этом размер вознаграждения напрямую зависит от вида страхования.

Проблематика заключается в том, что в страховой компании учёт и обработка данных ведутся разрозненно в локальных таблицах и на бумажных носителях. Это приводит к ошибкам и дублированию информации при ручном вводе, задержкам в оформлении и проверке договоров из-за переключения между разными источниками данных, отсутствию прозрачного и оперативного контроля за финансовыми показателями филиалов и начислением комиссионных агентов, а также к нарушениям целостности и безопасности данных в связи с отсутствием единой системы разграничения прав доступа. Всё это снижает скорость обслуживания клиентов, увеличивает вероятность ошибок и затрудняет принятие управленческих решений.

В качестве решения предлагается разработать клиент-серверное desktop-приложение на Java с использованием JavaFX для пользовательского интерфейса и MySQL для централизованного хранения данных. Приложение будет спроектировано по шаблону MVC, обеспечит единую базу данных, нормализованную в соответствии с нормальной формой Бойса-Кодда, и гарантирует целостность данных с помощью ключей, каскадных и ограничивающих операций. Параметрические SQL-запросы защитят от SQL-инъекций. В системе будет реализована регистрация и авторизация пользователей с ролями: администратор, агент и клиент, а также гибкий CRUD-функционал. Автоматический расчёт комиссионных агентов на основе тарифных ставок и страховых сумм упростит начисление вознаграждений. Проект будет вестись в удалённом Git-репозитории и собираться в исполняемый JAR-файл.

1 Постановка задачи

Цель работы — разработать клиент-серверное desktop-приложение с графическим интерфейсом на базе Java и JavaFX, обеспечивающее пользователю в соответствии с его ролью доступ к операциям чтения, добавления, изменения и удаления данных, хранящихся в СУБД MySQL, и моделирующих бизнес-процессы страховой компании.

Задачи:

1. Анализ предметной области и проектирование базы данных в СУБД MySQL: определение сущностей и их атрибутов, создание инфологической модели, разработка первичной реляционной модели с добавлением ключей, нормализация до нормальной формы Бойса-Кодда, формирование физической схемы с учётом целостности данных и связей между таблицами.

2. Настройка среды разработки: создание Maven-проекта на языке Java, подключение драйвера JDBC и библиотеки JavaFX, инициализация удалённого репозитория в Git.
3. Создание архитектуры приложения по шаблону MVC: разработка моделей, контроллеров и представлений. Реализация обёртки запросов в объект специально разработанных классов согласно шаблону Singleton. Создание параметрических запросов для защиты от SQL-инъекций.
4. Реализация системы регистрации и авторизации пользователей: разработка механизма создания учётных записей и проверки логина, обеспечение возможности изменения регистрационных данных.
5. Внедрение ролевой модели разграничения прав доступа: определение ролей (администратор, агент, клиент), настройка прав и реализация соответствующего CRUD-функционала в интерфейсе для каждой роли.
6. Разработка графического интерфейса на JavaFX: создание окон и элементов управления для отображения, добавления, редактирования и удаления данных по всем таблицам.
7. Тестирование и отладка: проверка корректности работы всех модулей, целостности данных, прав доступа, устранение ошибок.
8. Сборка и развертывание решения: ведение удалённого репозитория в Git, создание исполняемого JAR-файла.

2 Проектирование и разработка приложения

Приложение реализует клиент-серверную архитектуру и предоставляет функциональные возможности в соответствии с ролевой моделью доступа. Пользователь может зарегистрироваться, указав логин, пароль и роль, а затем пройти авторизацию. После входа в систему каждый

пользователь имеет возможность редактировать свои регистрационные данные. Каждый пользователь может выполнять поиск и фильтрацию записей таблиц по различным параметрам.

Администратор получает полный доступ ко всем таблицам базы данных. Он может просматривать, создавать, изменять и удалять записи в таблицах пользователей, клиентов, агентов, филиалов, видов страхования и договоров.

Агент может просматривать и редактировать свои личные данные, а также имеет доступ только к своим клиентам и договорам. Ему предоставляются возможности выполнять CRUD-операции с этими записями. Кроме того, агент может просматривать справочную информацию по видам страхования и филиалам.

Клиент имеет возможность просматривать и редактировать свои личные данные, а также просматривать справочную информацию по своим заключённым договорам.



Рисунок 1 – Диаграмма вариантов использования приложения

На рисунке 1 представлена диаграмма вариантов использования приложения, отражающая основные сценарии взаимодействия пользователей с системой.

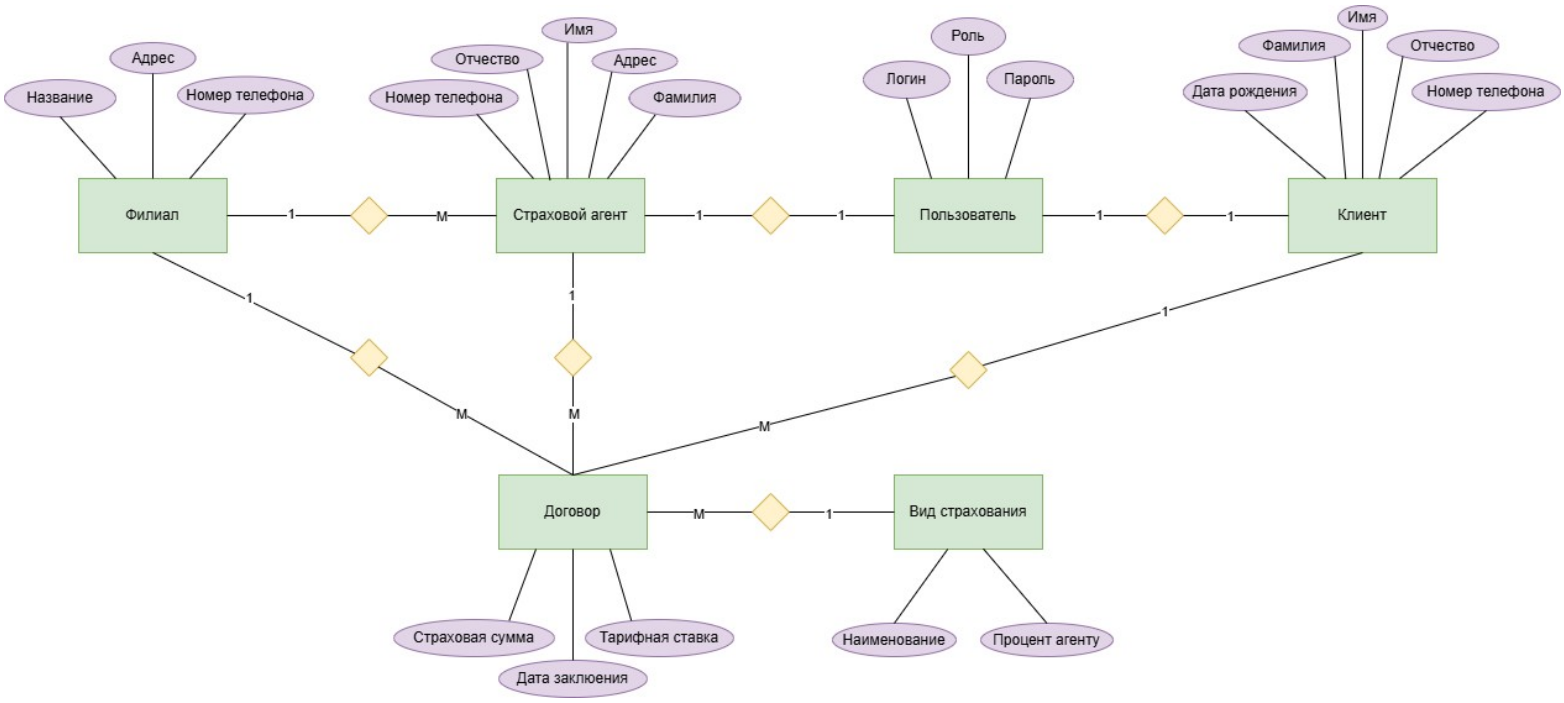


Рисунок 2 – Инфологическая модель базы данных

По описанию предметной области была разработана инфологическая модель в нотации Чена (рисунок 2). В модели выделены следующие основные сущности и их атрибуты: Пользователь (логин, пароль, роль), Филиал (название, адрес, номер телефона), Клиент (Фамилия, Имя, Отчество, дата рождения, номер телефона), Страховой агент (Фамилия, Имя, Отчество, адрес, номер телефона), Договор (дата заключения, страховая сумма, тарифная ставка), Вид страхования (наименование, тарифная ставка, процент агенту)

Данные сущности образуют следующие связи:

Страховой агент — Филиал (М:1). Страховой агент работает одновременно только в одном филиале. В филиале числится несколько страховых агентов.

Договор — Клиент (M:1). В договоре участвует только один клиент. Клиент может заключать множество договоров.

Договор — Страховой агент (M:1). Договор оформляет только один страховой агент. Страховой агент оформляет множество договоров.

Договор — Филиал (M:1). Договор оформляется в одном филиале. В филиале оформляется множество договоров.

Договор — Вид страхования (M:1). Договор оформляется на один вид страхования. На один и тот же вид страхования может оформляться множество договоров.

Пользователь — Страховой агент (1:1). Пользователь (владелец личного кабинета) может быть только одним страховым агентом. Агент может иметь единственный личный кабинет (быть одним пользователем).

Пользователь — Клиент (1:1). Пользователь (владелец личного кабинета) может быть только один клиент. Клиент может иметь единственный личный кабинет (быть одним пользователем).

На основе инфологической модели была построена реляционная модель (рисунок 3) базы данных, в которой каждая сущность и связь преобразованы в отдельные отношения. Для обеспечения однозначной идентификации записей использованы суррогатные первичные ключи. Связи между таблицами реализованы с помощью внешних ключей, соответствующих типам отношений. Те отношения, которые имели связь «один к одному» были соединены по общему атрибуту. Но несмотря на эту связь между отношениями «Пользователи», «Клиенты» и «Агенты», они не были объединены в одну таблицу, так как выполняют разные функции в системе. «Пользователи» хранит данные для авторизации, а «Клиенты» и «Агенты» представляют разные типы физических лиц. В системе один пользователь может быть либо клиентом, либо агентом, но не обоими одновременно. Объединение этих сущностей привело бы к необходимости хранения большого количества NULL-полей для

неиспользуемых атрибутов. Разделение обеспечивает корректную структуру и поддерживает целостность данных.

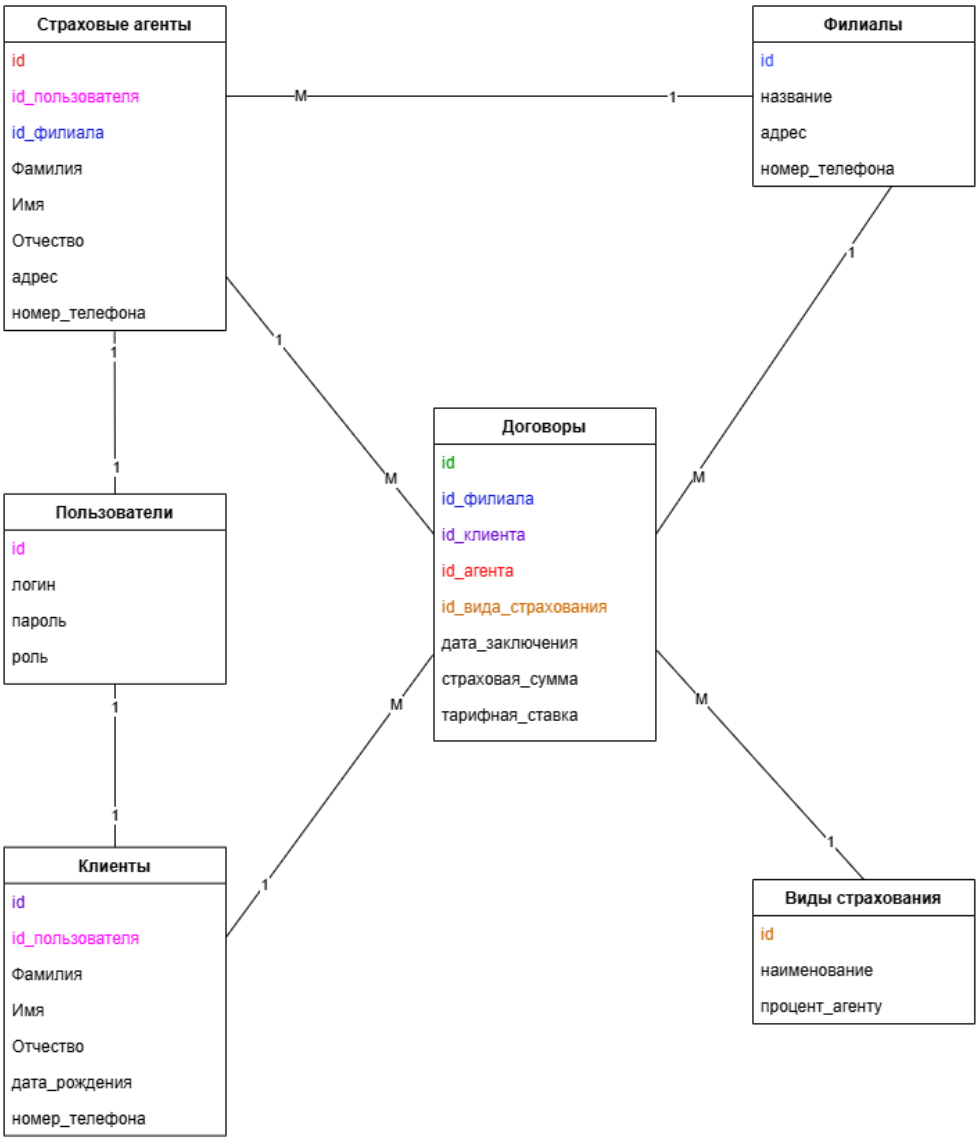


Рисунок 3 – Реляционная модель базы данных

В ходе проектирования реляционной модели базы данных была проведена проверка всех отношений на соответствие нормальной форме Бойса–Кодда. Для каждой таблицы установлено, что все функциональные зависимости являются зависимостями от потенциального ключа, ни один неключевой атрибут не зависит транзитивно или частично от составного ключа. Таким образом, все отношения удовлетворяют требованиям НФБК.

На основе разработанной реляционной модели была создана физическая модель базы данных, реализованная в среде СУБД MySQL. Для создания структуры таблиц, определения связей и ограничений, а также для заполнения базы данных тестовой информацией использовался инструмент phpMyAdmin, обеспечивающий удобный графический интерфейс для работы с MySQL. Физическая модель включает в себя таблицы, отражающие основные сущности и взаимосвязи, предусмотренные в рамках предметной области.

На основе разработанной реляционной модели была создана физическая модель базы данных в СУБД MySQL (рисунок 4). Для построения схемы использовался графический интерфейс phpMyAdmin, в котором определены все таблицы и связи. Полный текст SQL-скрипта создания структур таблиц приведён в приложении 1, там содержатся операторы CREATE TABLE с описанием полей, первичных и внешних ключей, а также правил каскадного и ограничивающего удаления.

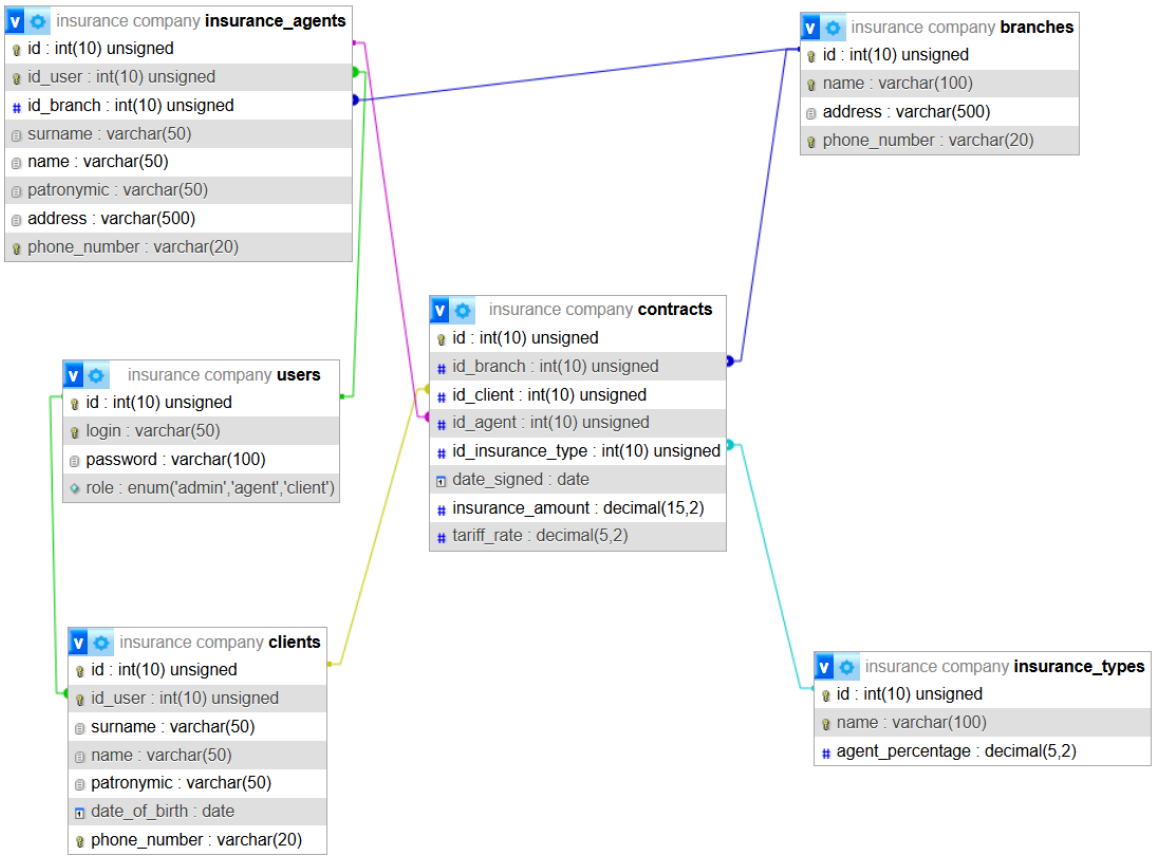


Рисунок 4 – Физическая модель базы данных

В состав физической модели входят таблицы:

Таблица Users хранит учётные записи всех пользователей системы. Каждая запись содержит уникальный идентификатор, логин и пароль для авторизации, а также роль — администратор, агент или клиент.

Таблица Clients предназначена для хранения персональных данных клиентов (ФИО, дата рождения, номер тедефона) страховой компании. Она связана с Users через внешний ключ, который гарантирует, что каждая учётная запись соответствует именно одному клиенту.

Таблица Branches отражает структуру сети филиалов компании. Для каждого филиала сохраняются название, полный адрес и контактный телефон. Эти данные используются при оформлении договоров и распределении агентов по филиалам.

Таблица Insurance_Agents содержит информацию о страховых агентах (ФИО, адрес, номер телефона), также связанных с учётными записями в Users и закреплённых за конкретным филиалом через внешний ключ.

Таблица Insurance_Types описывает доступные виды страхования и хранит их наименования вместе с процентной ставкой вознаграждения агента. Это позволяет гибко рассчитывать комиссионные в зависимости от выбранного вида страхования.

Таблица Contracts фиксирует все заключённые страховые договоры и связывает их с конкретным филиалом, клиентом, агентом и видом страхования. Здесь же хранят дату подписания, страховую сумму и применённую тарифную ставку. Ограничения на удаление предотвращают потерю связанных данных и обеспечивают целостность истории договоров.

Далее была разработано приложение, программный код которого состоит из 37 классов, организованных в соответствии с архитектурным шаблоном Model-View-Controller. Для каждой из шести таблиц разработаны по два класса модели (запись и контейнер таблицы), по одному контроллеру и одному представлению. Дополнительные модули реализуют механизм регистрации и

авторизации пользователей (регистрационный и входной контроллеры и представления), а также три набора контроллер-представление для личных кабинетов администратора, агента и клиента. В состав проекта входят класс Main и класс ConnectionDatabase, ответственные за запуск приложения и установление соединения с СУБД MySQL, а также общий интерфейс для моделей таблиц, определяющий методы загрузки, сохранения и обновления данных.

Класс Main служит точкой входа в приложение. Метод main() запускает JavaFX-платформу, а в start() настраивается и отображается главное окно, устанавливается начальная сцена входа или регистрации и создаётся соединение с базой данных через ConnectionDatabase для загрузки пользователей в модель. Все ошибки инициализации интерфейса и подключения обрабатываются, после чего управление передаётся соответствующему контроллеру в зависимости от роли пользователя.

Класс ConnectionDatabase отвечает за установку и хранение соединения с MySQL-базой данных. В его конструкторе по заданной строке подключения создаётся объект JDBC Connection, доступный через публичное поле connection. В случае ошибки при подключении генерируется сообщение об ошибке.

Интерфейс TableModel предназначен для унификации работы с различными моделями таблиц базы данных в приложении страховой компании. Он определяет стандартный набор операций CRUD (создание, чтение, обновление, удаление), а также метод для загрузки данных из базы.

В приложении реализован ряд классов (User, Client, Agent, Branch, InsuranceType, Contract), представляющих записи таблиц базы данных Эти классы используются для хранения, отображения и обработки информации, связанной с основными сущностями предметной области. Каждый класс включает конструктор и стандартные методы доступа (геттеры и сеттеры), обеспечивающие работу с полями. В классе Contract реализованы вспомогательные методы getPaymentString()

и getSalaryString() для вычисления суммы страхового платежа и агентского вознаграждения.

В приложении создан набор классов-обёрток (UsersTable, ClientsTable, AgentsTable, BranchesTable, InsuranceTypesTable, ContractsTable), реализованных по шаблону Singleton, они работают с таблицами базы данных, каждая из которых отвечает за конкретную сущность предметной области. Все классы реализуют интерфейс TableModel<T>, обеспечивающий единый подход к выполнению операций добавления, удаления, обновления, поиска и загрузки данных. Каждый из классов содержит коллекцию объектов соответствующего типа, синхронизированную с базой данных. Реализованы основные методы: loadFromDatabase() — загрузка данных из базы, add(), remove(), update() — операции добавления, удаления и изменения данных, findById() — поиск по идентификатору, getAll() — получение полного списка.

В приложении реализован ряд контроллеров (UserTableController, ClientsTableController, AgentsTableController, BranchesTableController, InsuranceTypesTableController, ContractsTableController), каждый из которых отвечает за управление таблицей определённой сущности в графическом интерфейсе. Контроллеры реализуют обработку всех пользовательских действий: добавление handleAdd(), редактирование handleEdit(), удаление handleDelete(), поиск updateFilter(), загрузка данных из базы load…FromModel(), а также отображение ошибок showError(String message). Каждый контроллер содержит ссылку на соответствующую модель и представление, а также коллекции ObservableList и FilteredList, которые используются для отображения и динамической фильтрации данных в таблице. В конструкторах контроллеров осуществляется связывание модели и представления, а также настройка всех обработчиков событий.

В приложении реализован набор классов представлений (UserTableView, ClientTableView, AgentsTableView, BranchesTableView, ContractsTableView, InsuranceTypesTableView), отвечающих за построение графического интерфейса для отображения и управления таблицами. Все классы представлений наследуются от

VBox, что обеспечивает единый вертикальный макет для таблицы, кнопок и элементов фильтрации. Каждое представление содержит поле table типа TableView, отображающее соответствующие записи. Кнопки управления представлены полями addBtn, editBtn и deleteBtn, которые позволяют выполнять базовые операции. Для фильтрации данных предусмотрены поле поиска searchField и комбинированный список filterColumnBox. В конструкторах классов происходит создание и настройка колонок таблицы, элементов управления и их размещение в графическом интерфейсе. Для взаимодействия с контроллерами все представления содержат методы setAddButtonHandler, setEditButtonHandler и setDeleteButtonHandler, позволяющие назначить обработчики событий для соответствующих кнопок.

Класс EntranceView отвечает за построение интерфейса окна входа. В конструкторе сохраняется переданный объект Stage, вызывается приватный метод createView(), в котором создаются и размещаются элементы интерфейса: текстовое поле loginField для ввода логина, поле passwordField для ввода пароля, кнопки loginBtn («Войти») и regBtn («Регистрация») и метка message для вывода системных сообщений. Методы setLoginButtonHandler() и setRegisterButtonHandler() позволяют назначить контроллеру обработчики нажатий кнопок.

Класс RegistrationView строит окно создания нового пользователя. В конструкторе вызывается createView(), создающий текстовое поле loginField, поле passwordField, выпадающий список roleCombo для выбора роли пользователя (Администратор, Агент, Клиент), кнопку submitBtn («Зарегистрироваться») и метку message для обратной связи. Метод setSubmitButtonHandler() назначает обработчик кнопки регистрации, а метод show() отображает окно.

Класс EntranceController связывает модель пользователей UsersTable и представление EntranceView. В конструкторе назначаются внутренние обработчики: LoginButtonHandler, который нажатие кнопки «Войти» обрабатывает получением loginField и passwordField, поиском пользователя в модели по логину, в зависимости от его роли (admin, agent, client) открывает соответствующий личный кабинет или

выводит сообщение об ошибке; и RegisterButtonHandler, который открывает окно регистрации при нажатии кнопки «Регистрация».

Класс RegistrationController управляет логикой окна регистрации, связывая UsersTable и RegistrationView. В конструкторе назначается SubmitButtonHandler, который при нажатии «Зарегистрироваться» считывает логин, пароль и выбранную роль, приватный метод register(), проверяющий уникальность логина, добавляющий запись в базу. В случае успеха на экране появляется сообщение об успешной регистрации и пользователь возвращается к окну входа, в противном случае выводится ошибка.

Класс AdminViewController отвечает за управление логикой личного кабинета администратора. Он хранит ссылку на представление AdminView. В конструкторе происходит связывание с представлением и назначение обработчика выбора таблицы через метод setTableSelectorHandler(). Внутренний класс TableSelectorHandler в методе handle(ActionEvent) обрабатывает выбор элемента в ComboBox tableSelector: в зависимости от строки-значения он заменяет содержимое корневого контейнера соответствующим таблицам.

Класс AgentViewController управляет логикой личного кабинета агента. Он хранит поля view (AgentView), agent (Agent) и user (User). В конструкторе сначала вызываются приватные методы fillProfile() и loadAgentTables() для заполнения текстовых полей профиля и загрузки в таблицы клиентов агента, его договоров, всех филиалов и видов страхования. Затем назначаются обработчики: SaveProfileHandler для сохранения изменений профиля агента, EditUserHandler для открытия окна изменения логина и пароля, SaveUserHandler для сохранения новых регистрационных данных.

Класс ClientViewController реализует управление личным кабинетом клиента. Он хранит поля view (ClientView), client (Client) и user (User). В конструкторе вызываются методы fillProfile() для заполнения полей профиля клиента и loadClientContracts() для загрузки только его договоров в contractsTableView. Затем

назначаются обработчики SaveProfileHandler для сохранения изменений профиля, EditUserHandler для открытия окна изменения регистрационных данных и SaveUserHandler для их сохранения.

Класс AdminView реализует графический интерфейс личного кабинета администратора. Его поля: stage (Stage) — главное окно, user (User) — текущий администратор, root (VBox) — корневой контейнер, tableSelector (ComboBox<String>) для выбора таблицы и шесть представлений таблиц. В конструкторе вызываются методы createTableSelector(), createModels(), createTableViews(), createControllers() и createMainInterface() для построения выпадающего списка, инициализации моделей и представлений, создания контроллеров и компоновки элементов интерфейса. Метод show() отображает окно, а setTableSelectorHandler() назначает обработчик выбора таблицы.

Класс AgentView реализует графический интерфейс личного кабинета агента. Его публичные поля surnameField, nameField, patronymicField, addressField и phoneField используются для показа и редактирования профиля. Поля saveBtn и editUserBtn — кнопки сохранения профиля и открытия окна изменения пользователя. Поля clientTableView, contractsTableView, branchesTableView и insuranceTypesTableView — представления таблиц. В конструкторе вызываются приватные методы createProfileUI() для создания блока профиля, и createSectionSelectorAndTables() для построения выпадающего списка разделов sectionSelector и настройки переключения таблиц в методе updateTableView(). Метод show(String title) отображает окно с заголовком title, а публичные методы setSaveProfileHandler(), setEditUserHandler() и др. позволяют назначить обработчики на кнопки и выпадающий список.

Класс ClientView реализует графический интерфейс личного кабинета клиента. Его публичные поля surnameField, nameField, patronymicField, dobField и phoneField служат для отображения и редактирования профиля клиента. Поля saveBtn и editUserBtn — кнопки сохранения профиля и изменения пользователя. Поле contractsTableView — представление списка договоров клиента. В

конструкторе вызывается приватный метод createProfileUI() для блока профиля и инициализируется contractsTableView. Метод show(String title) отображает окно, а публичные методы setSaveProfileHandler() и setEditUserHandler() назначают обработчики для кнопок.

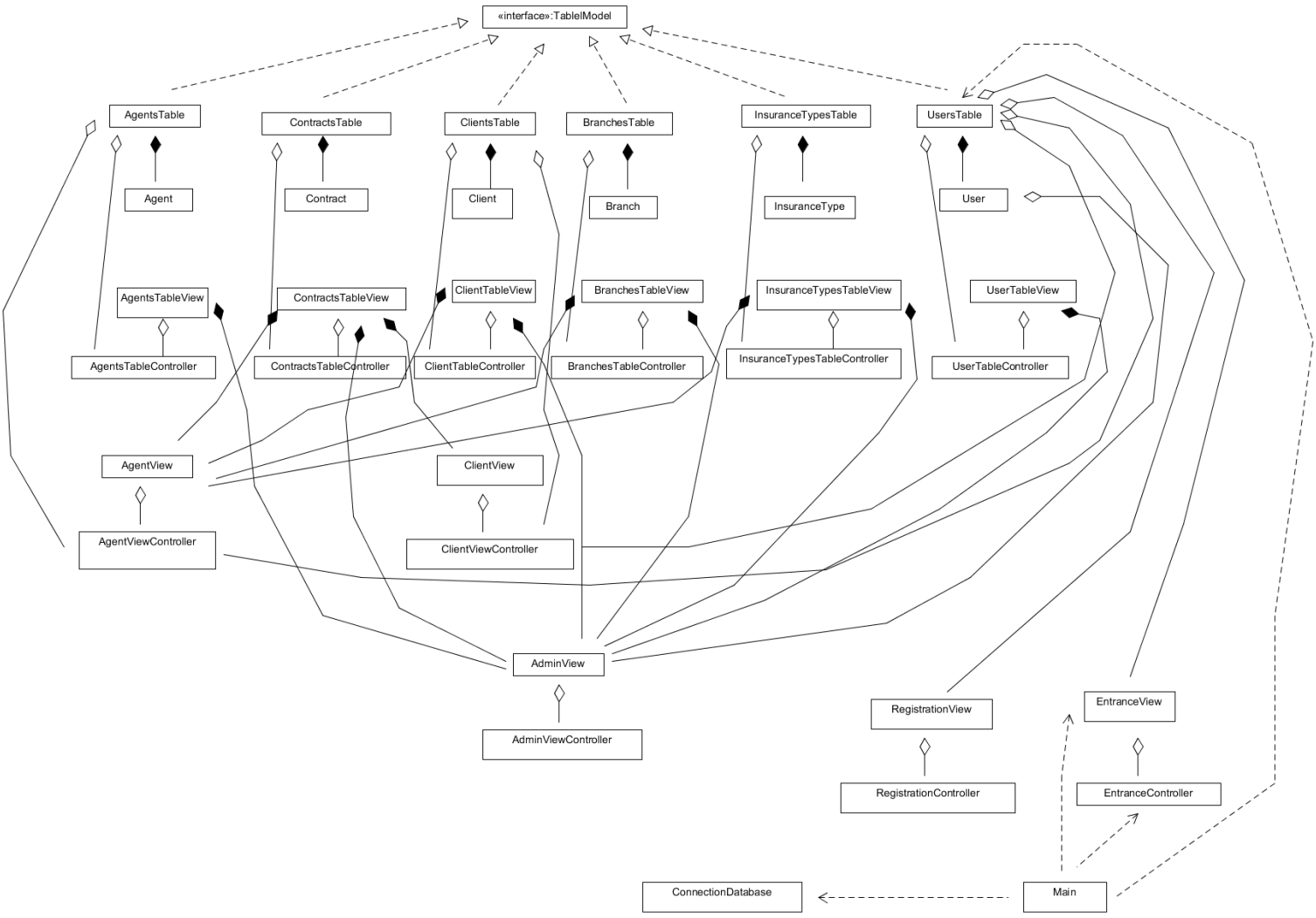


Рисунок 5 – Диаграмма классов

На рисунке 5 представлена диаграмма классов, демонстрирующая структуру приложения, его ключевые компоненты, а также взаимосвязи между ними. Диаграмма отображает основные модели, контроллеры и представления, соответствующие архитектурному шаблону MVC.

Рассмотрим некоторые методы классов:

Метод loadFromDatabase предназначен для загрузки всех записей из соответствующей таблицы базы данных в память приложения. Он используется во всех классах моделей, реализующих интерфейс TableModel. Метод обеспечивает актуализацию внутренней коллекции объектов, синхронизируя её с текущим состоянием базы данных. Он принимает объект подключения к базе данных типа Connection и выполняет SQL-запрос SELECT * для получения всех записей из нужной таблицы. Перед загрузкой коллекция очищается, чтобы исключить дублирование данных. Затем с помощью объекта ResultSet выполняется построчное считывание результатов запроса. Из каждой строки извлекаются значения полей, на их основе создаётся экземпляр соответствующего класса-модели, который добавляется в коллекцию. В конце метод закрывает все используемые ресурсы.

```
public void loadFromDatabase(java.sql.Connection conn) throws java.sql.SQLException {
    users.clear();
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM Users");

    while (rs.next()) {
        int id = rs.getInt("id");
        String login = rs.getString("login");
        String password = rs.getString("password");
        String role = rs.getString("role");

        User user = new User(id, login, password, role);
        users.add(user);
    }

    rs.close();
    stmt.close();
}
```

Метод updateFilter служит для динамической фильтрации отображаемых в TableView данных на основе введённого пользователем текста поиска и выбранного столбца для фильтрации. Он реализуется во всех контроллерах таблиц и позволяет без обращения к базе данных, скрывать или показывать строки в зависимости от

соответствия содержимого ячеек условиям поиска. Метод не принимает параметров и выполняет следующие действия. Сначала он считывает текущее значение поля поиска searchField, приводит его к нижнему регистру и сохраняет в локальную переменную filter. Затем получает название выбранного столбца filterColumnBox и сохраняет его в переменную column. После этого метод вызывает filteredData.setPredicate, передавая ему новую реализацию интерфейса Predicate. Внутри предиката реализован метод test, который для каждого объекта модели проверяет: если строка поиска пуста, возвращается true и все элементы отображаются; иначе, в зависимости от значения column, сравнивает строку filter с соответствующим полем объекта. В случае выбора опции «Все» предикат объединяет проверки по всем основным полям через логическое ИЛИ. Любой объект, для которого test возвращает true, остаётся в отфильтрованном списке и отображается в таблице, остальные — скрываются.

```
private void updateFilter() {
    final String filter = view.searchField.getText().toLowerCase();
    final String column = view.filterColumnBox.getValue();
    filteredData.setPredicate(new Predicate<User>() {
        public boolean test(User user) {
            if (filter == null || filter.isEmpty()) return true;
            if ("ID".equals(column)) return String.valueOf(user.getId()).contains(filter);
            if ("Логин".equals(column)) return user.getLogin().toLowerCase().contains(filter);
            if ("Пароль".equals(column)) return user.getPassword().toLowerCase().contains(filter);
            if ("Роль".equals(column)) return user.getRole().toLowerCase().contains(filter);

            return String.valueOf(user.getId()).contains(filter)
                || user.getLogin().toLowerCase().contains(filter)
                || user.getPassword().toLowerCase().contains(filter)
                || user.getRole().toLowerCase().contains(filter);
        }
    });
}
```

Методы add, update и remove, реализованные во всех классах-обёртках моделей таблиц, предназначены для управления записями как в базе данных, так и в

соответствующих коллекциях в памяти приложения. Они описывают логику формирования и выполнения параметризованных SQL-запросов.

Метод add(T obj, Connection conn) создаёт новую запись в таблице базы данных и добавляет соответствующий объект в коллекцию. С помощью конструкции INSERT INTO формируется SQL-запрос с подстановочными знаками вопроса для всех необходимых полей. Затем открывается PreparedStatement, в который устанавливаются параметры. Вызовом executeUpdate выполняется запрос, после чего через getGeneratedKeys извлекается первичный ключ. Если ключ получен, он присваивается полю id объекта. Statement и ResultSet закрываются, а объект помещается в коллекцию модели.

```
    public void add(User user, java.sql.Connection conn) throws java.sql.SQLException {
        String sql = "INSERT INTO Users(login, password, role) VALUES (?, ?, ?)";
        try (PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS))
{
            stmt.setString(1, user.getLogin());
            stmt.setString(2, user.getPassword());
            stmt.setString(3, user.getRole());
            stmt.executeUpdate();
            ResultSet rs = stmt.getGeneratedKeys();
            if (rs.next()) {
                user.setId(rs.getInt(1));
            }
            rs.close();
            users.add(user);
        }
    }
```

Метод update(T obj, Connection conn) обновляет существующую запись в базе данных и синхронизирует изменения в коллекции. Он формирует запрос UPDATE с параметрами для изменяемых полей и для идентификатора записи в WHERE-условии. Подготовленный PreparedStatement получает новые значения из полей и текущее значение id. После выполнения executeUpdate SQL-команды коллекция уже содержит обновлённый объект.

```
public void update(User user, java.sql.Connection conn) throws java.sql.SQLException {
    String sql = "UPDATE Users SET login = ?, password = ?, role = ? WHERE id = ?";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, user.getLogin());
        stmt.setString(2, user.getPassword());
        stmt.setString(3, user.getRole());
        stmt.setInt(4, user.getId());
        stmt.executeUpdate();
    }
}
```

Метод remove(T obj, Connection conn) удаляет запись из базы и удаляет объект из коллекции. Он формирует DELETE FROM с параметром id, открывает PreparedStatement, устанавливает значение id и выполняет executeUpdate. После успешного удаления из базы объект удаляется из внутреннего списка модели с помощью users.remove(obj).

```
public void remove(User user, java.sql.Connection conn) throws java.sql.SQLException {
    String sql = "DELETE FROM Users WHERE id = ?";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, user.getId());
        stmt.executeUpdate();
```

```
    }
    users.remove(user);
}
```

3 Тестирование и сценарии работы в приложении

Тест-кейс 1: регистрация и вход в систему.

Пользователь запускает приложение (см. рис. 6), переходит на форму регистрации, вводит желаемые логин и пароль (см. рис. 7) и подтверждает создание учётной записи. После успешной регистрации окно закрывается, и пользователь возвращается на форму авторизации. Он вводит вновь созданные данные для входа и подтверждает. Система принимает введённые учётные данные, выполняет вход и отображает главное окно приложения (см. рис. 8).

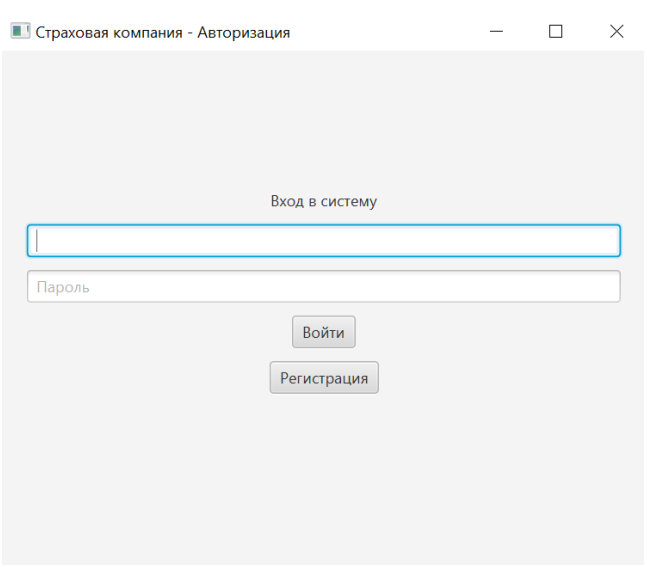


Рисунок *1* – Окно авторизации

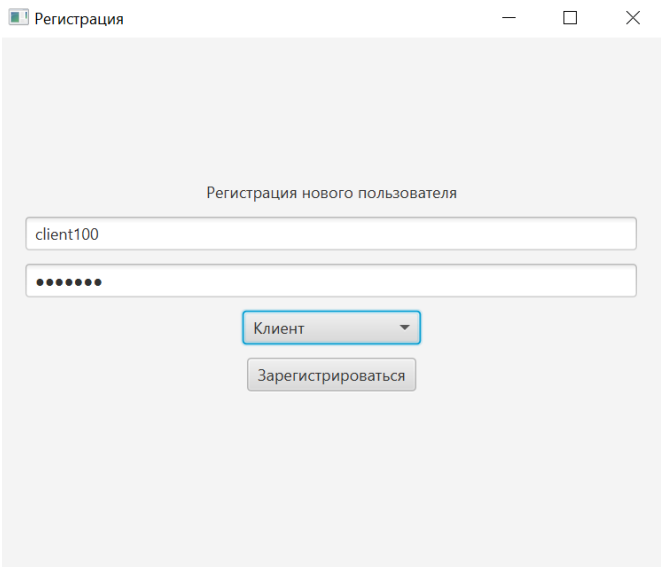


Рисунок 7 – Окно регистрации

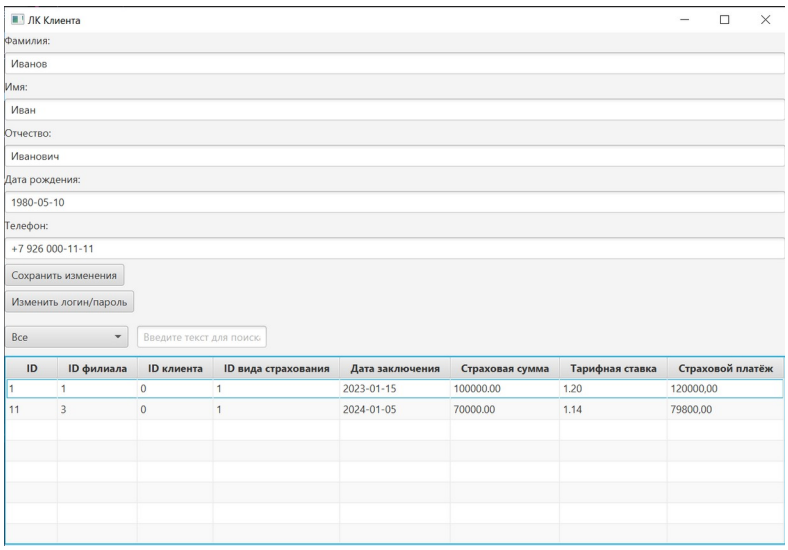


Рисунок 8 – Открытие личного кабинета

Тест-кейс 2. Добавление, изменение и удаление нового филиала:

Пользователь (с ролью администратора) авторизируется и входит в систему (рис. 9). Далее выбирает таблицу филиалы, нажимает на кнопку «Добавить» и вводит данные (рис. 10). Сохраняет запись. Новая запись появляется в таблице, данные корректно отображаются (рис. 11). Потом администратор решает изменить номер телефона нажав на кнопку «Изменить» (рис. 12). Изменения сохраняются и применяются (рис. 13). Далее удаляем новую запись, нажав на кнопку «Удалить», запись успешно удаляется (рис. 14).

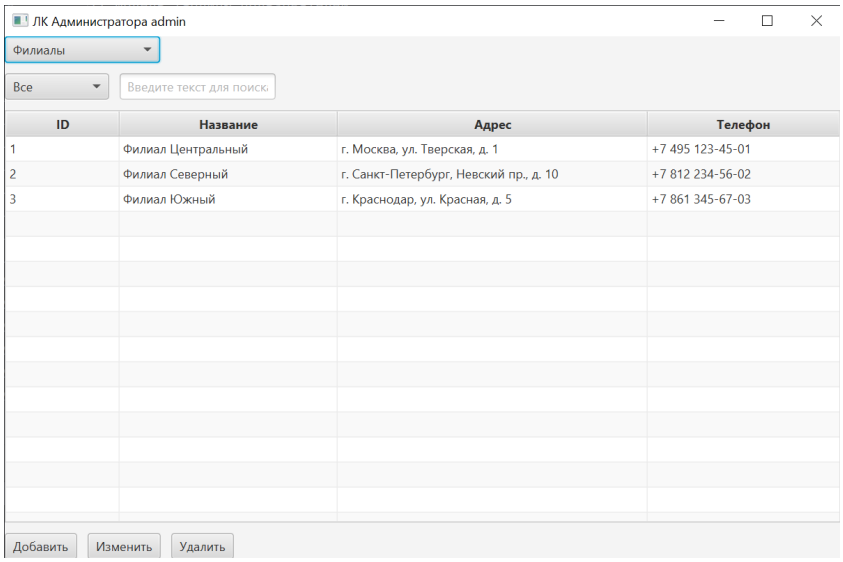


Рисунок 9 — Личный кабинет администратора, таблица Филиалы

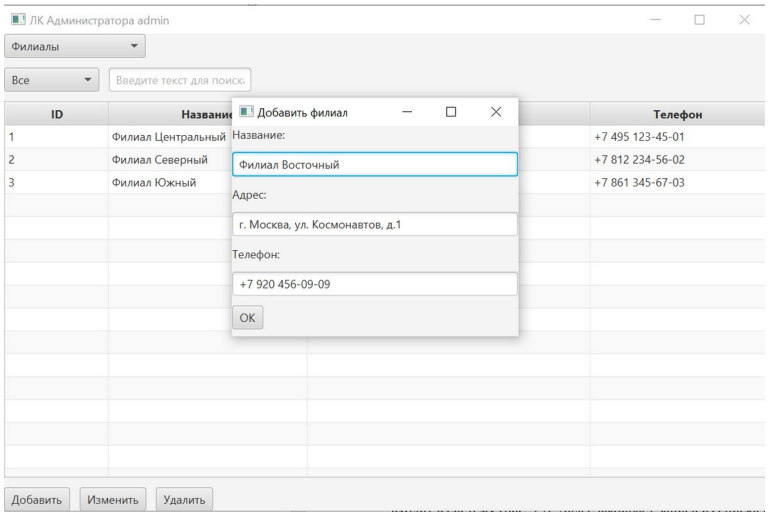


Рисунок 10 — Добавление нового филиала в таблицу Филиалы

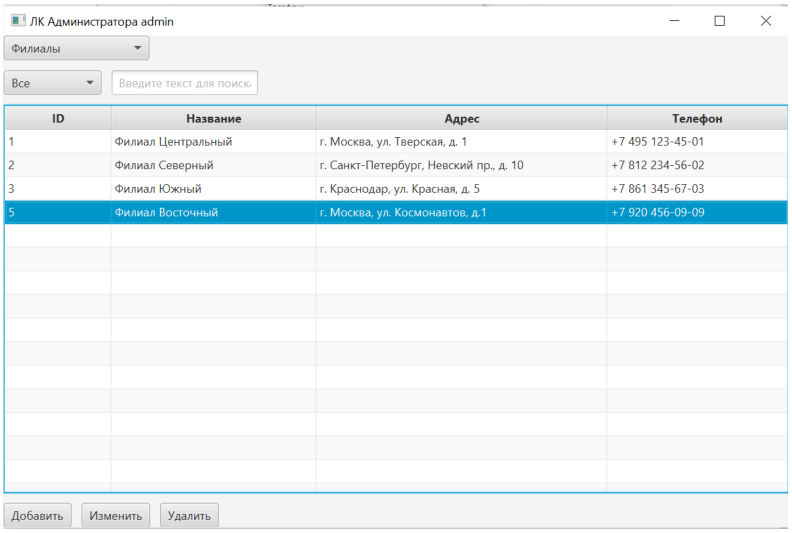


Рисунок 11 – Появление записи в таблице

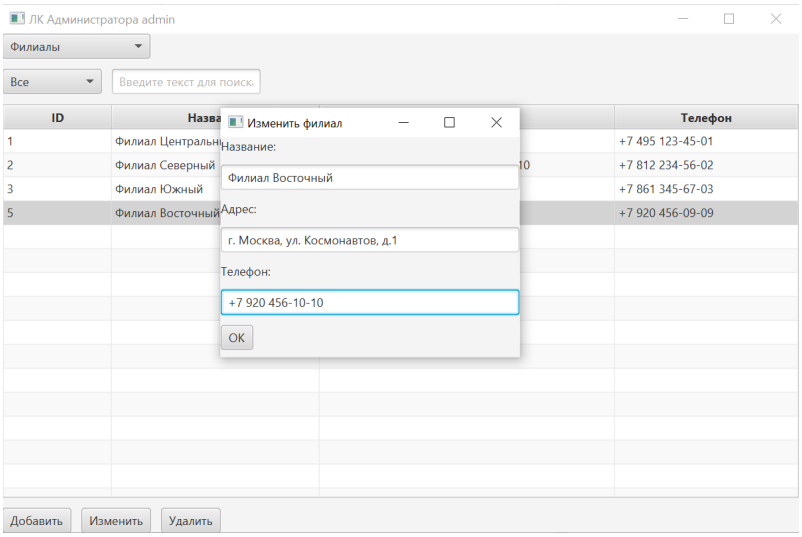


Рисунок 2 — Изменение номера телефона филиала

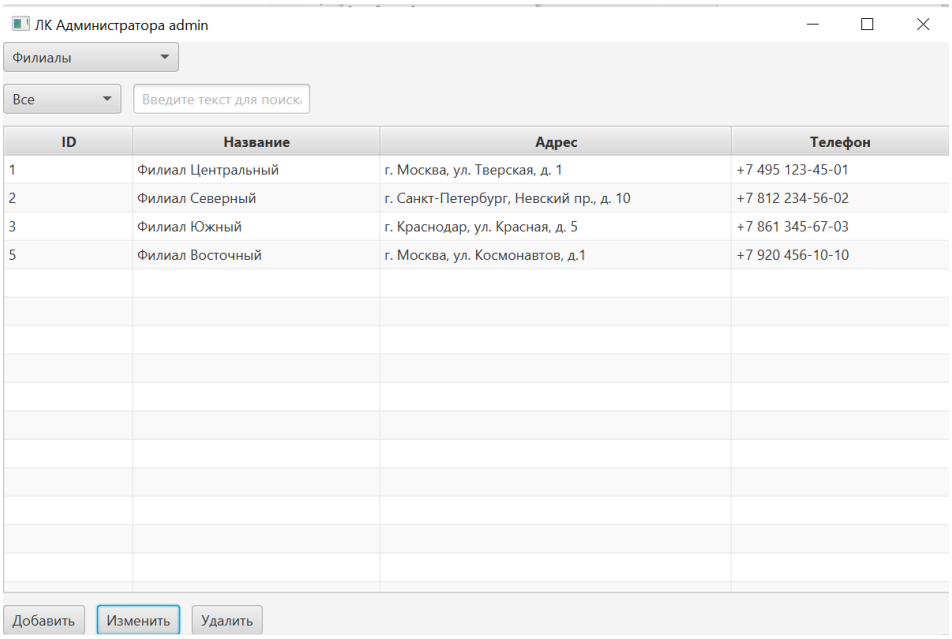


Рисунок 13 – Отображение изменённых данных в таблице

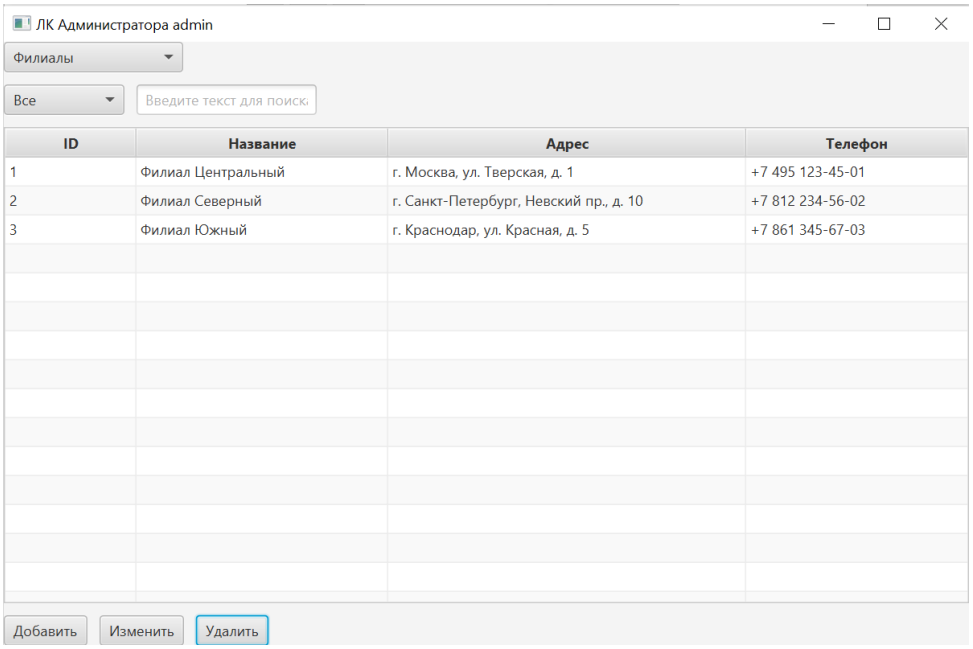


Рисунок 14 – Запись удалена

Тест-кейс 3. Фильтрация и поиск данных:

Пользователь (с ролью страхового агента) авторизируется и входит в свой личный кабинет (рис. 15). Далее в таблице Мои клиенты выбирает фильтр на фамилию и пытается найти «Смирнову» (рис. 16). Записи успешно фильтруются.

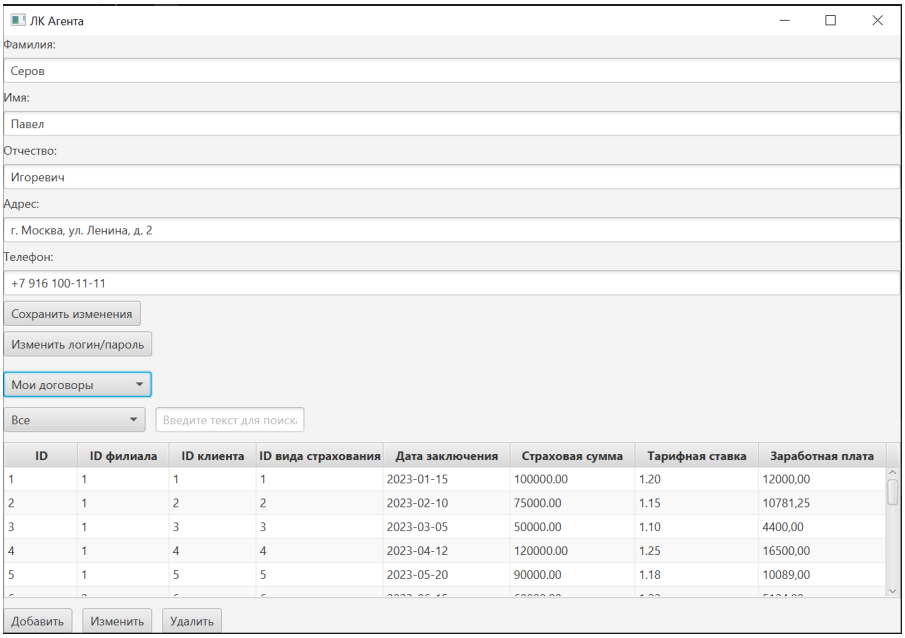


Рисунок 15 – Личный кабинет страхового агента

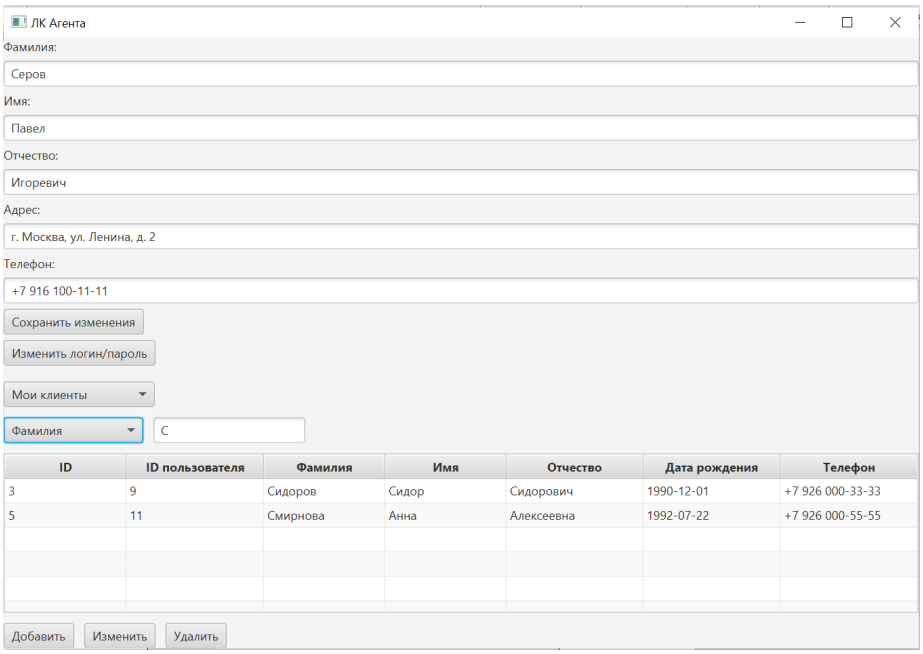


Рисунок 16 – Поиск по фамилии

Заключение

В результате выполнения курсового проекта было разработано клиент-серверное desktop-приложение для поддержки бизнес-процессов страховой компании. Решение реализовано на языке Java с использованием JavaFX для графического интерфейса и MySQL для централизованного хранения данных. Приложение построено по архитектурному шаблону MVC и обеспечивает ролевую модель доступа (администратор, агент, клиент), полный CRUD-функционал, регистрацию и авторизацию пользователей, а также автоматический расчёт комиссионных агентов.

Ключевыми преимуществами системы являются единая база данных с нормализованной схемой и строгой целостностью, надёжное разграничение прав доступа, интуитивно понятный графический интерфейс и высокая отзывчивость благодаря локальной фильтрации и поиску. Использование параметризованных SQL-запросов минимизирует риск SQL-инъекций.

Среди недостатков можно отметить относительно простой внешний вид интерфейса и ограничение только десктопной платформой без веб- или мобильной версии.

В дальнейшем проект может быть расширен за счёт добавления аналитических и отчётных модулей, улучшения дизайна интерфейса, а также разработки веб- и мобильных клиентов для более гибкого доступа к системе.

Исходный код и история изменений доступны в репозитории https://github.com/Maria-Git2025/-2-.

Список литературы и интернет-ресурсов

1. Java Tutorials Learning Paths [Электронный ресурс]. URL: https://docs.oracle.com/javase/tutorial/tutorialLearningPaths.html (дата обращения: 02.02.2025).
2. Курс «Базы данных» в СДО Московского Политеха [Электронный ресурс]. URL: https://online.mospolytech.ru/course/view.php?id=14740 (дата обращения: 10.02.2025).
3. Курс «Прикладное программирование» в СДО Московского Политеха [Электронный ресурс]. URL: https://online.mospolytech.ru/course/view.php?id=15500 (дата обращения: 06.04.2025).
4. Официальная документация JavaFX [Электронный ресурс]. URL: https://openjfx.io/ (дата обращения: 20.06.2025).
5. Руководство по работе с MySQL [Электронный ресурс]. URL: https://dev.mysql.com/doc/ (дата обращения: 20.04.2025).

Приложения

Приложение 1

```
CREATE TABLE Users (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    login VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL,
    role ENUM('admin', 'agent', 'client') NOT NULL
);

CREATE TABLE Clients (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    id_user INT UNSIGNED UNIQUE NOT NULL,
    surname VARCHAR(50) NOT NULL,
    name VARCHAR(50) NOT NULL,
    patronymic VARCHAR(50),
    date_of_birth DATE NOT NULL,
    phone_number VARCHAR(20) UNIQUE NOT NULL,
    FOREIGN KEY (id_user) REFERENCES Users(id) ON DELETE CASCADE
);

CREATE TABLE Branches (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) UNIQUE NOT NULL,
    address VARCHAR(500) NOT NULL,
    phone_number VARCHAR(20) UNIQUE NOT NULL
);
```

```
CREATE TABLE Insurance_Agents (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    id_user INT UNSIGNED UNIQUE NOT NULL,
    id_branch INT UNSIGNED NOT NULL,
    surname VARCHAR(50) NOT NULL,
    name VARCHAR(50) NOT NULL,
    patronymic VARCHAR(50),
    address VARCHAR(500) NOT NULL,
    phone_number VARCHAR(20) UNIQUE NOT NULL,
    FOREIGN KEY (id_user) REFERENCES Users(id) ON DELETE CASCADE,
    FOREIGN KEY (id_branch) REFERENCES Branches(id) ON DELETE CASCADE
);

CREATE TABLE Insurance_Types (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) UNIQUE NOT NULL,
    agent_percentage DECIMAL(5, 2) NOT NULL
);

CREATE TABLE Contracts (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    id_branch INT UNSIGNED NOT NULL,
    id_client INT UNSIGNED NOT NULL,
    id_agent INT UNSIGNED NOT NULL,
```

```
    id_insurance_type INT UNSIGNED NOT NULL,
    date_signed DATE NOT NULL,
    insurance_amount DECIMAL(15, 2) NOT NULL,
    tariff_rate DECIMAL(5, 2) NOT NULL,
    FOREIGN KEY (id_branch) REFERENCES Branches(id) ON DELETE RESTRICT,
    FOREIGN KEY (id_client) REFERENCES Clients(id) ON DELETE RESTRICT,
    FOREIGN KEY (id_agent) REFERENCES Insurance_Agents(id) ON DELETE RESTRICT,
    FOREIGN KEY (id_insurance_type) REFERENCES Insurance_Types(id) ON DELETE RESTRICT
);
```